

Taller RMI

Introducción a sistemas distribuidos



Juan Carlos Santamaria Orjuela

Juan Felipe Gutiérrez adarme

Juan Daza

Juan Rincón

Pontificia Universidad Javeriana

Facultad de ingeniería

Departamento de ingeniería de sistemas

Bogotá D.C.

2026

1 introducción	2
2 Objetivos del Taller	3
2.1 Objetivo General	3
2.2 Objetivo Especifico	3
2.3 Alcance del taller.....	4
3 Arquitectura del Sistema	5
3.1 Arquitectura Fisica	5
3.2 Arquitectura Logica	6
3.3 Modelo de Comunicacion	8
3.4 Flujo General de Ejecucion	8
3.4 Diseño de componentes del sistema distribuidos	8
3.4.1COMPONENTE BASE DE DATOS	9
3.4.2 COMPONENTE SERVIDOR.....	10
3.4.3 COMPONENTE CLIENTE	19
4 diseño del servicio gRPC.....	20
4.1 Definicion del servicio gRPC	20
4.2 Definición de Servicio Remoto	21
4.3 Diseño de los Mensajes	21
4.4 Generacion deCodigo Automatico	21
4.5 Tipo de Comunicacion Implementada	22
4.6 Justificacion del uso de gRPC.....	22
5 Diseño de la Base de Datos	23
5.1 Justificación del modelo Relacional.....	23
5.2 Modelo General de Entidades	23
5.3 Diseño de la Entidad Usuario	23
Campos principales:.....	23
5.4 Diseño de la entidad Libro	24
Campos principales:.....	24
5.5 Diseño de la Entidad Prestamo	24
Campos principales:.....	24
5.6 Integridad Referencial y Restricciones	25
5.7 Optimizacion e indices	25
5.8 Configuracion para Acceso Remoto	26

6 Implementacion del Servicio	26
6.1 Estructura y Configuracion del taller	26
6.2 Arquitectura Interna por Capas	26
ServerMain	27
Capa Service	27
Capa Business.....	27
Capa Repository	28
Capa DB	28
Capa Util	28
6.3 Flujo de Procesamiento de una Solicitud	28
6.4 Concurrencia y Robustez	29
7 Implementacion del Cliente	30
7.1 Entorno y Configuracion del Cliente.....	30
7.2 Conexion al Servidor	30
7.3 Invocacion de Metodos Remotos	30
7.4 Flujo de Comunicacion Cliente-Servidor	31
7.5 Manejo de Errores en el Cliente	32
7.6 Interoperabilidad entre Lenguajes	33
8 Pruebas y validacion del sistema.....	33
8.1 Verificacion de Infraestructura y Conectividad de Red	33
8.2 Inicializacion y Disponiibilidad del Servidor gRPC	34
8.3 Validacion del Modelo Relacional en PostgreSQL	34
8.4 Prueba Funcionales del Servicio Remoto	35
8.5 Validacion de Reglas de Negocio	37
8.6 Pruebas de Manejo de errores y robustez	37
8.7 Verificacion de Persisntencia de Datos.....	37
8.8 Resultados Generales	37
9 Conclusiones.....	37
10 Referencias	37

1 introducción

En la actualidad, los sistemas distribuidos constituyen la base de gran parte de la infraestructura tecnológica moderna, desde servicios en la nube hasta aplicaciones empresariales que operan sobre múltiples nodos interconectados. A diferencia de los sistemas centralizados tradicionales, un sistema distribuido se caracteriza por la ejecución de componentes en diferentes máquinas que se comunican entre sí a través de la red para cumplir un objetivo común. Esta separación física implica desafíos relacionados con conectividad, sincronización, consistencia de datos y comunicación remota entre procesos.

En el contexto académico de la asignatura **Introducción a Sistemas Distribuidos**, el presente **TallerRMI-2026-10** tiene como finalidad aplicar de manera práctica los fundamentos teóricos estudiados en clase, mediante el diseño e implementación de un sistema distribuido funcional. El sistema desarrollado permite la gestión de préstamos de libros utilizando una arquitectura cliente-servidor, en la cual múltiples máquinas virtuales interactúan a través de comunicación remota estructurada creando diferentes roles entre los nodos o pcs, los cuales son 3 el primero es el servidor donde estará toda la lógica del negocio y el que conectara los servicios del cliente a la lógica y a la base de datos, el segundo es la base de datos la cual mantiene la información actualizada y consistente para el servidor y sus clientes, luego esta el rol cliente el cual actúa como el que solicita los servicios a partir de las funciones del programa.

Para lograr este objetivo, se implementó un servidor desarrollado en Java 17 que expone servicios remotos mediante gRPC, framework que permite la invocación de procedimientos remotos (RPC) sobre el protocolo HTTP/2. Los clientes fueron desarrollados en Python, demostrando interoperabilidad entre lenguajes distintos mediante el uso de Protocol Buffers como mecanismo de serialización y definición de contrato formal de comunicación. La persistencia de la información se gestiona a través de una base de datos PostgreSQL alojada en una máquina virtual independiente, reforzando la separación física de componentes.

El taller no se limita únicamente al desarrollo de funcionalidades, sino que también aborda aspectos fundamentales de un entorno distribuido real, tales como la configuración de conectividad entre nodos, apertura de puertos, configuración de acceso remoto a la base de datos y validación de comunicación entre procesos ubicados en diferentes máquinas. De esta manera, se busca comprender no solo el

funcionamiento lógico del sistema, sino también los elementos de infraestructura que permiten su operación.

A lo largo del documento se describe la arquitectura física y lógica del sistema, el diseño del servicio gRPC, la estructura de la base de datos, la implementación del servidor y los clientes, así como las pruebas realizadas para validar su correcto funcionamiento. Finalmente, se presentan conclusiones que permiten reflexionar sobre los aprendizajes obtenidos y las posibles mejoras futuras.

Este taller representa un primer acercamiento práctico al desarrollo de sistemas distribuidos, consolidando conocimientos relacionados con comunicación remota, separación de responsabilidades, persistencia de datos y coordinación entre múltiples componentes en un entorno heterogéneo implementando un grid ósea una red de nodos interconectados trabajando para compartir y coordinar recursos entre varias máquinas.

2 Objetivos del Taller

2.1 Objetivo General

Implementar y desarrollar un sistema distribuido para administrar préstamos de libros mediante gRPC como medio de comunicación remota entre procesos, integrando varias máquinas virtuales con responsabilidades específicas y utilizando varios lenguajes de programación para evidenciar interoperabilidad en ambientes heterogéneos.

El sistema tiene como objetivo demostrar la implementación apropiada de una arquitectura distribuida cliente-servidor, en la que un servidor desarrollado en Java 17 se encarga de procesar las solicitudes hechas desde clientes programados en Python. Este servidor interactúa con una base de datos PostgreSQL situada en una máquina virtual autónoma. Se asegura la compatibilidad entre lenguajes, la tipificación fuerte y la eficiencia a través de la serialización con Protocol Buffers y el protocolo HTTP/2 en los componentes.

2.2 Objetivo Especifico

- Diseñar e implementar un servicio remoto utilizando gRPC que permita ejecutar operaciones de préstamo, consulta y devolución de libros de manera síncrona.
- Establecer una arquitectura distribuida compuesta por cuatro máquinas virtuales independientes: dos clientes, un servidor y una base de datos, cada una con una dirección IP propia y responsabilidades separadas.
- Implementar una separación clara por capas en el servidor, dividiendo el sistema en:
 - Capa de comunicación (gRPC)
 - Capa de lógica de negocio
 - Capa de acceso a datos (Repository)
 - Capa de conexión a base de datos
 - Capa de manejo de excepciones
- Diseñar un modelo relacional en PostgreSQL que permita gestionar usuarios, libros y préstamos mediante claves primarias, foráneas y reglas de integridad referencial.
- Configurar la base de datos para permitir conexiones remotas seguras desde el servidor, ajustando adecuadamente los archivos postgresql.conf y pg_hba.conf.
- Demostrar la interoperabilidad entre lenguajes (Java y Python) mediante el uso de un archivo .proto que actúa como contrato formal de comunicación entre cliente y servidor.
- Validar el correcto funcionamiento del sistema a través de pruebas de conectividad, ejecución de métodos remotos y verificación del estado de la base de datos tras cada operación.

2.3 Alcance del taller

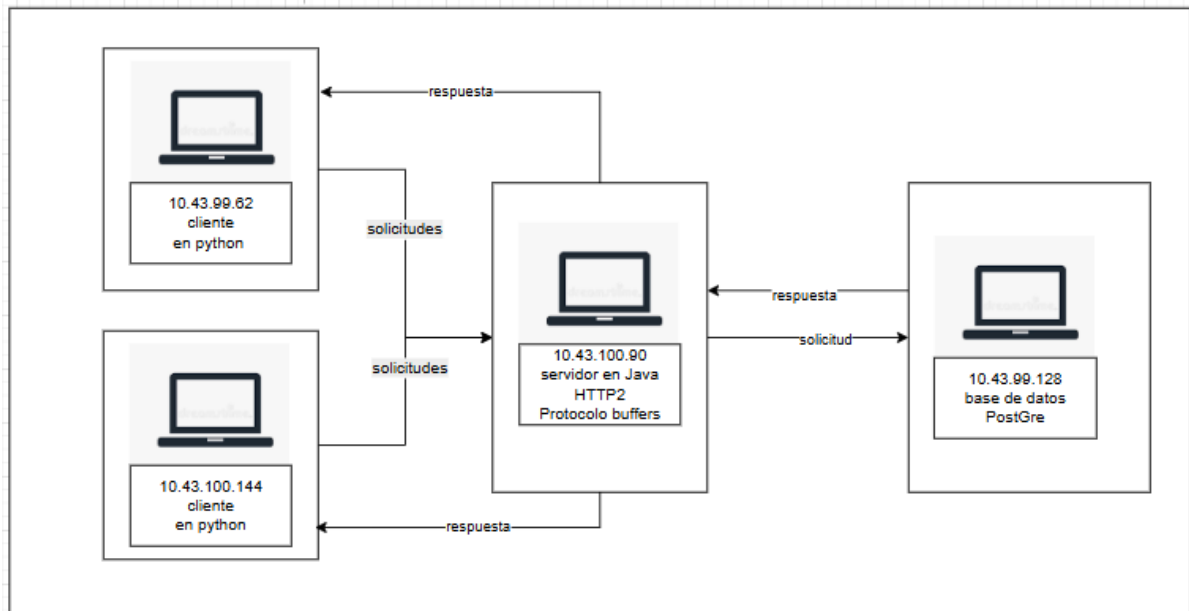
El sistema desarrollado permite la gestión básica de una biblioteca digital mediante operaciones remotas síncronas. Se contemplan funcionalidades diferenciadas para roles de administrador y cliente, donde el cliente puede realizar préstamos y consultas, y el administrador puede gestionar información del sistema según las reglas de negocio definidas.

El sistema implementa concurrencia porque la base de datos es centralizada y transaccional los clientes pueden acceder a los mismos datos. Ambos llaman al mismo servidor Java que utiliza gRPC este servidor escribe en la base de datos y lee desde la fuente de datos Postgre. Cualquier cambio que haga un cliente queda guardado en la base de datos y es visible para los demás. Esto también es gracias a que en Postgre se implementan principios como ACID persistencia consistencia y aislamiento de las diferentes transacciones como un semáforo. El servidor no guarda datos en memoria todo es directamente con la base de datos. Aprovecha el modelo de ejecución de gRPC sobre HTTP/2 y el manejo interno de conexiones del servidor para procesar solicitudes remotas de manera estructurada y ordenada. El modelo gRPC es sincrónico sobre un servidor multihilo para cada uno de los clientes se abre un hilo distinto el cual permite disponibilidad de las dos máquinas activas en el servidor al mismo tiempo. El servidor es sincrónico y hay paralelismo se permiten ejecuciones simultáneas en diferentes núcleos o sea el servidor tiene varios hilos y la máquina del sistema tiene varios cores por lo cual se permiten hacer diferentes transacciones al mismo tiempo. Es implícito se maneja desde el modelo gRPC y Postgre. También se implementan bloqueos desde el pc de postgre y en el código se activan desde la capa de repository que se encarga de las consultas sql. También como el sistema funciona a partir de gRPC funciona a partir de las peticiones HTTP2 con el protobuf binario (serialización de datos) cuando se activa el servidor y el puerto 50051 confirma que el servidor está funcionando y el protocolo está activo. Por cada llamada del cliente internamente se abre la conexión HTTP2 la cual envía los datos al protobuf y el servidor con datos serializados en formato protobuf en formato binario. Por medio de la librería del pom Netty se implementa el protocolo http2 que está montado en la librería de gRPC. Lo único que se define a nivel de archivo es el .proto que define las funciones de los clientes y los datos que se van a usar en las transacciones. El archivo que recibe las llamadas es BibliotecaServiceImpl.java en el cual no hay http2 solo se recibe los objetos instanciados en Java no se ven ni los headers (información de los paquetes) ni las requests HTTP2. El protocolo HTTP/2 no se configura manualmente. Es manejado automáticamente por la librería grpc-netty incluida en el pom.xml. El servidor se inicia en ServerMain.java mediante ServerBuilder, lo que crea un servidor HTTP/2 sobre el puerto 50051. El sincronismo funciona a partir de aislamiento commits y bloqueos pero no hay un reloj global incorporado.

3 Arquitectura del Sistema

3.1 Arquitectura Física

Para complementar se hizo la arquitectura del negocio en un diagrama.



Fuente: hecho en visual studio se puede consultar en:

https://drive.google.com/file/d/1q7sZrHeSVOOGqTbYYc3S-_fQVLjl40tP/view?usp=sharing

La arquitectura física del sistema fue diseñada bajo un enfoque distribuido real, utilizando cuatro máquinas virtuales independientes desplegadas sobre entorno Linux (Ubuntu 22.04). Cada máquina cumple una responsabilidad específica dentro del sistema, garantizando separación de funciones y simulando un escenario de infraestructura empresarial distribuida.

La distribución física está compuesta por:

- Máquina Virtual 1: Cliente Python 1
- Máquina Virtual 2: Cliente Python 2
- Máquina Virtual 3: Servidor gRPC desarrollado en Java 17
- Máquina Virtual 4: Servidor de base de datos PostgreSQL 14

Cada máquina posee una dirección IP propia dentro de la red interna, permitiendo comunicación remota entre procesos mediante el uso de sockets TCP/IP. El servidor gRPC escucha solicitudes entrantes en el puerto 50051 utilizando el protocolo HTTP/2, mientras que la base de datos PostgreSQL opera sobre el puerto 5432, configurado para aceptar conexiones remotas desde la máquina donde reside el servidor Java.

Esta separación física permite evidenciar un verdadero sistema distribuido, donde los componentes no comparten memoria ni espacio de ejecución, y toda interacción se realiza a través de comunicación remota estructurada. Además, este enfoque permite analizar problemas reales como conectividad, configuración de firewall, apertura de puertos y permisos de acceso, elementos fundamentales en arquitecturas distribuidas modernas.

3.2 Arquitectura Logica

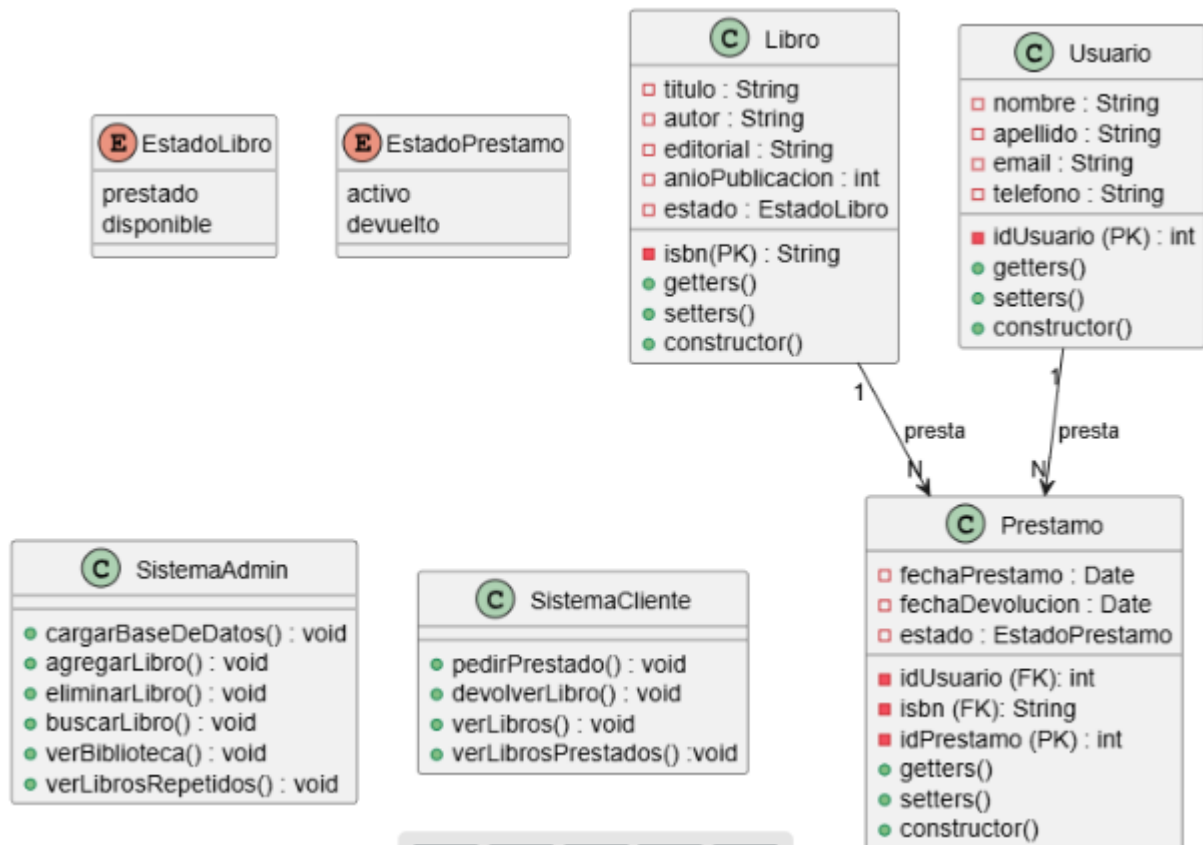
Desde el punto de vista lógico, el sistema implementa una arquitectura cliente-servidor basada en el modelo de invocación remota de procedimientos (RPC). Los clientes desarrollados en Python actúan como consumidores de servicios, mientras que el servidor Java implementa la lógica central del sistema y actúa como intermediario entre los clientes y la base de datos.

La arquitectura lógica del servidor está organizada en múltiples capas claramente diferenciadas, siguiendo principios de separación de responsabilidades:

- Capa de Comunicación (gRPC):
Encargada de recibir las solicitudes remotas, deserializar los mensajes Protocol Buffers y delegar la ejecución a la capa de negocio.
- Capa de Lógica de Negocio:
Implementa las reglas del sistema, valida condiciones, controla estados de libros y préstamos, y gestiona el flujo transaccional.
- Capa Repository (Acceso a Datos):
Contiene las consultas SQL y operaciones CRUD necesarias para interactuar con la base de datos.
- Capa de Conexión a Base de Datos:
Gestiona la conexión JDBC hacia PostgreSQL, administrando la apertura y cierre de conexiones.
- Capa de Manejo de Excepciones:
Centraliza el tratamiento de errores técnicos y de negocio, garantizando respuestas controladas al cliente.

Este diseño modular permite mejorar la mantenibilidad, escalabilidad y claridad estructural del sistema, además de facilitar pruebas unitarias y futuras ampliaciones.

Ademas se hizo un diseño de base de datos



Fuente: hecho en visual studio se puede consultar en:

<https://drive.google.com/file/d/1q7sZrHeSVOOGqTbYYc3S-fQVLjl40tP/view?usp=sharing>

Para complementar el flujo del sistema es recibir la petición del cliente que se hace con el protocolo gRPC por medio del protobuf que serializa los datos, y del archivo.proto que sirve como contrato para conectar lenguajes de programación entre cliente y servidor. Luego de ver la petición se valida el usuario como admin o cliente, después se inicia la transacción se hace el llamado al repositorio y en el repositorio al momento de hacer la consulta si hay un cliente simultáneo y intenta la misma transacción la base de datos bloquea uno de los dos clientes para evitar interbloqueo entre los clientes.

3.3 Modelo de Comunicación

El sistema implementa comunicación remota utilizando gRPC como framework principal de RPC. gRPC opera sobre el protocolo HTTP/2, lo cual permite multiplexación de conexiones, comunicación eficiente y bajo consumo de recursos en comparación con enfoques tradicionales basados en HTTP/1.1.

La definición del contrato de comunicación se realiza mediante un archivo .proto, el cual describe formalmente los servicios disponibles, los métodos remotos y las estructuras de datos intercambiadas entre cliente y servidor. Este archivo actúa como un contrato

independiente del lenguaje, permitiendo generar automáticamente código cliente en Python y código servidor en Java a partir de una misma definición.

La serialización de datos se realiza mediante Protocol Buffers, un formato binario compacto y fuertemente tipado que garantiza eficiencia en transmisión y compatibilidad entre diferentes lenguajes de programación. Gracias a esta característica, el sistema demuestra interoperabilidad real entre entornos heterogéneos.

El modelo de comunicación implementado es síncrono, es decir, el cliente envía una solicitud y permanece en espera hasta recibir la respuesta correspondiente del servidor. Este enfoque facilita la trazabilidad de operaciones y simplifica el control del flujo lógico en un entorno académico.

3.4 Flujo General de Ejecucion

El flujo operativo del sistema puede describirse de la siguiente manera:

1. El cliente Python invoca un método remoto definido en el archivo .proto.
2. La solicitud es serializada mediante Protocol Buffers.
3. El mensaje viaja a través de HTTP/2 hacia el servidor Java.
4. El servidor recibe la solicitud en la capa gRPC.
5. La capa de negocio procesa la operación solicitada.
6. Si es necesario, se ejecutan consultas SQL en PostgreSQL mediante JDBC.
7. Se construye una respuesta estructurada.
8. La respuesta es serializada y enviada nuevamente al cliente.

Este flujo demuestra una arquitectura distribuida completa, donde cada componente cumple una función específica dentro de una cadena de procesamiento claramente definida.

3.4 Diseño de componentes del sistema distribuidos

Para ver en detalle el proceso acceda a este link donde se hizo toda la documentacion del proceso en un diagrama: [Componente Base de Datos](#)

3.4.1COMPONENTE BASE DE DATOS

El componente de base de datos mantiene los datos del actalogo de los libros de los usuarios y de la cuenta admin garantizando la consistencia cuando hay varios hilos al tiempo.

En sus componentes estan primero los enums los cuales estan los estados de libros y de los prestamos, en sus tablas estan libro prestamo y usuario permitiendo la gestion de las solicitudes.

```

biblioteca=# \dT
              List of data types
 Schema |      Name      | Description
-----+-----+-----
 public | estado_libro   |
 public | estado_prestamo |
(2 rows)

biblioteca=#

```

```

biblioteca=# \dt
              List of relations
 Schema |   Name   | Type  | Owner
-----+-----+-----+-----
 public | libro    | table | postgres
 public | prestamo | table | postgres
 public | usuario  | table | postgres
(3 rows)

```

En estas se definen reglas como constraints permitiendo por ejemplo que el año de publicacion no sea superior al año actual, tambien los no NULL para que no hayan datos vacios , tambien los cascade y restricts que mantienen actualizada las tablas que tienen relaciones entre ellas (foreign and primary keys) ademas se crean los index para hacer las consultas precisas de los clientes sin hacer consultas gigantes sino definir consultas repetitivas que buscan una tupla especifica. Para permitir la conexion adecuada a la base se instalo postgres, se definieron las tablas y enums se insertaron datos iniciales, se crearon las reglas de bloqueo, luego de eso se establecio la conexion con el servidor para ello desde el servidor en el proyecto de java se debe crear la conexion en la carpeta db en la cual estar toda la informacion de la ocnexion de la base de datos hay no se tratan datos internos de la base de datos solo de la conexión. Se define en localhost gracias a que estan en la misma red lan, se definen imports de la sesion de la base de datos, la creacion de conexion, y los errores de la abse de datos, luego se define la clase como Db y dentro de la clase se definen los parametros de conexión como el host para saber dónde esta postgres(localhost), el puerto del servicio osea 5432, el nombre de la base de datos el usuario que sera el intercesor de la base de datos y su contraseña y la funcion getConnection para generar la conexión y que se puedan hacer las transacciones. Con eso ya estaria configurada todala base de datos.

3.4.2 COMPONENTE SERVIDOR

Lo primero fue la creacion de la carpeta del proyecto

Descargando el jdk 17

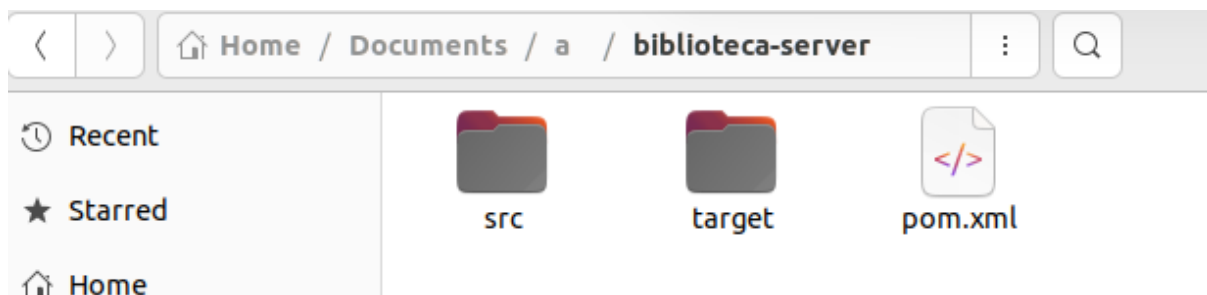
Luego egnerando la carpeta del proyecto y generando el arquetipo

En el cual se crea el src y el pom

Luego se añaden todas las dependencias plugins y herramientas y librerías que se usaran y se crea en el src la carpeta proto y un archivo librería..proto donde se definen las transacciones o servicios que posea el servidor. Luego se compila y maven automáticamente crea la carpeta target donde estará todo el ejecutable y herramientas para ejecutar de forma correcta el programa y servidor. A partir de eso se genera toda la lógica de negocio y se recompila en con mvn compile.

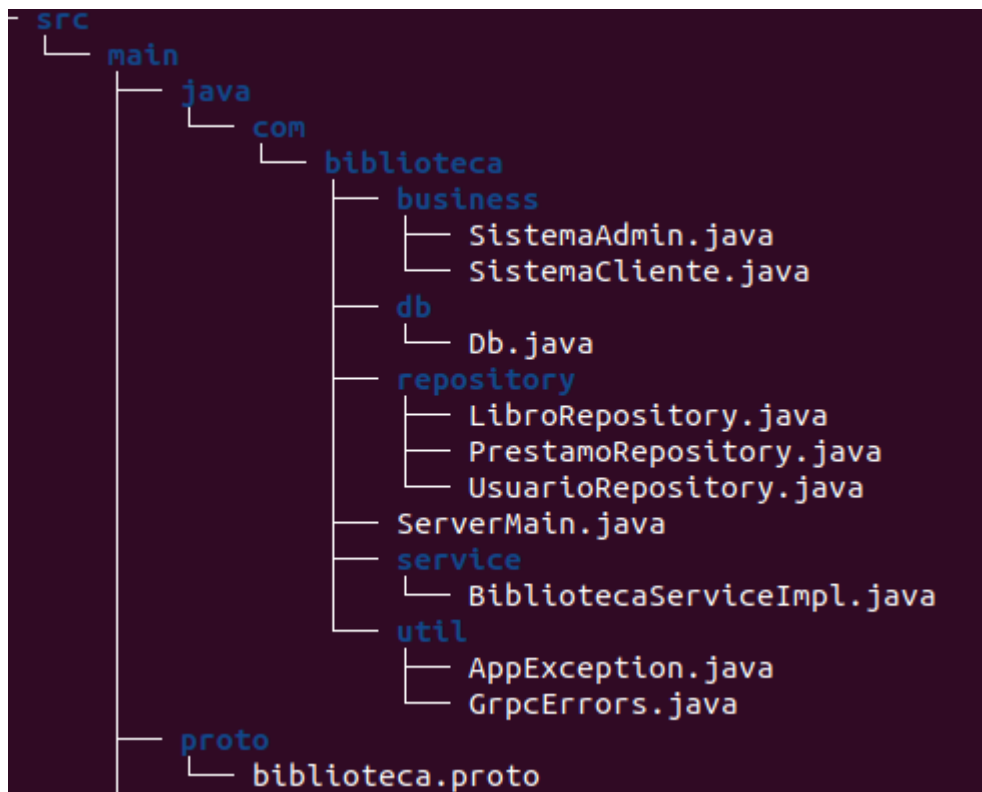
A continuación, la estructura de carpetas del servidor

Estructura de carpetas del código servidor:

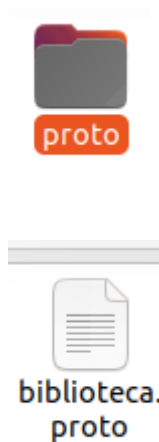


El src donde estará toda la lógica del negocio y esto debe ser lo único que tocamos nosotros como programadores los archivos externos se generan automáticamente con el pom instalando las dependencias y recursos necesarios para la ejecución correcta en java del proyecto.





En primer lugar tenemos la carpeta proto donde esta el contrato Biblioteca.proto



Biblioteca.proto define el contrato gRPC definiendo los servicios(funciones remotas) y mensajes(los datos que estaran usandose), sus funciones se dividen en las de rol admin y cliente las de cliente son:

- VerLibros(Vacio) -> ListaLibros: lista toda la biblioteca.
- BuscarLibro(LibroIdRequest) -> Libro: busca por ISBN.
- PrestarLibro(PrestamoRequest) -> Respuesta: presta un libro.
- DevolverLibro(DevolucionRequest) -> Respuesta: devuelve por idPrestamo.

- VerLibrosPrestados(UsuarioRequest) -> ListaPrestamos: préstamos activos de un usuario

Las de admin son:

- CargarBaseDeDatos(AdminVacioRequest) -> Respuesta
- AgregarLibro(LibroAdminRequest) -> Respuesta
- EliminarLibro(LibroIdAdminRequest) -> Respuesta
- VerBiblioteca(AdminVacioRequest) -> ListaLibros
- VerLibrosRepetidos(AdminVacioRequest) -> ListaLibros

Y los mensajes o datos que serán usados.

```

69 message ListaLibros {
70   repeated Libro libros = 1;
71 }
72
73 // ----- Usuario -----
74 message UsuarioRequest {
75   int32 idUsuario = 1;
76 }
77
78 // ----- Préstamo -----
79 message PrestamoRequest {
80   int32 idUsuario = 1;
81   string isbn = 2;
82 }
83
84 message DevolucionRequest {
85   int32 idPrestamo = 1;
86 }
87
88 message Prestamo {
89   int32 idPrestamo = 1;
90   int32 idUsuario = 2;
91   string isbn = 3;
92   string fechaPrestamo = 4;
93   string fechaDevolucion = 5;
94   string estado = 6;
95 }
96
97 message ListaPrestamos {
98   repeated Prestamo prestamos = 1;
99 }

```



En la carpeta src se poseera toda la logica de el servidor



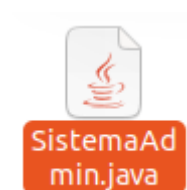
En esta carpeta estan las conexiones del servidor



En este archivo esta la conexion de la base de datos como explique arriba en el componente base de datos



Por otra parte estan los archivos de negocio que representan el sistema para el cliente y para el admin en estos:



Para **SistemaAdmin.java** estan las reglas de negocio del administrador, sus responsabilidades son validar admin con la adminkey(contraseña) traer de LibroRepository para ejecutar la base de datos. Sus metodos principales son:

- cargarBaseDeDatos(conn) → seedLibrosSiVacio un por si algo
- agregarLibro(conn, libro) → valida campos y hace insert

- `eliminarLibro(conn, isbn)` → valida y borra
- `verBiblioteca(conn)` → `findAll`
- `verLibrosRepetidos(conn)` → query de repetidos



Para **SistemaCliente.java** estan las reglas de negocio del usuario comun, sus responsabilidades son validar el usuario normal y controla los estados de los libros a partir de las acciones que haga el usuario sus metodos son

En detalle:

verLibros(conn):

Devuelve todo LOS ATRIBUTOS DE UN LIBRO.

buscarLibro(conn, isbn):

- Valida ISBN
- Busca en repo
- Si no existe genera `AppException.notFound`

verLibrosPrestados(conn, idUsuario):

- valida id
- check usuario existe
- busca préstamos activos

pedirPrestado(conn, idUsuario, isbn) (la más importante)

Flujo correcto para consistencia:

1. validar `idUsuario` + `isbn`
2. validar usuario existe
3. `libroRepo.getEstadoForUpdate` (bloquea fila del libro con `FOR UPDATE`)
4. si no disponible → `FAILED_PRECONDITION`
5. `prestamoRepo.insertPrestamoActivo`
 - a. si rompe por constraint → `FAILED_PRECONDITION`
6. `libroRepo.updateEstado(..., "prestado")`

7. retorna OK

devolverLibro(conn, idPrestamo)

1. valida idPrestamo
2. bloquea el préstamo activo con FOR UPDATE
3. marca devuelto y pone fecha
4. cambia estado del libro a disponible
- 5.

Por otra parte estan los archivos de repositorio que representan la persistencia de los datos libros usuarios y prestamos.



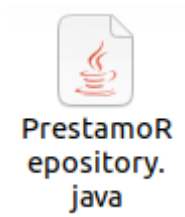
Aqui se enuentra el codigo LibroRepository.java



Este es la capa de acceso a datos de la tabla libros de la base de datos sus responsabilidades es ejecutar SQL haciendo operaciones CRUD (create update delete y read) sus metodos principales son:

- insertLibro: inserta con estado 'disponible'
- deleteLibro: borra por isbn
- findByIsbn: SELECT + construir Libro
- findAll: SELECT ORDER BY titulo + construir lista
- findRepetidosPorTituloAutor: query con GROUP BY HAVING COUNT>1
- getEstadoForUpdate: SELECT ... FOR UPDATE (bloqueo pesimista)
- updateEstado: UPDATE casteando a enum DB estado_libro
- seedLibrosSiVacio: inserta libros iniciales si tabla vacía

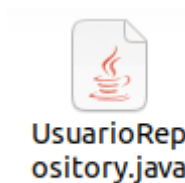
Aqui se enuentra el codigo PrestamoRepository.java



Este es la capa de acceso a datos de la tabla Prestamo de la base de datos sus responsabilidades es ejecutar SQL haciendo operaciones CRUD (create update delete y read) sus metodos principales son:

- insertPrestamoActivo: INSERT estado 'activo'
- findPrestamosActivosPorUsuario: SELECT WHERE estado='activo'
- getIsbnPrestamoActivoForUpdate: SELECT ... FOR UPDATE (bloquea ese préstamo)
- marcarDevuelto: UPDATE estado=devuelto + fecha_devolucion=CURRENT_DATE

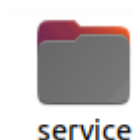
Aqui se enuentra el codigo UsuarioRepository.java



Este es la capa de acceso a datos de la tabla Prestamo de la base de datos sus responsabilidades es ejecutar SQL haciendo operaciones CRUD (create update delete y read) sus metodos principales es solo uno:

- existsById(conn, idUsuario)

Valida que exista el usuario.



Luego esta la carpeta service enfocado en ser el controlador de la implementacion de los datos y funciones del contrato o del archivo.proto



En ese documento se encuentra el archivo BibliotecaServicImpl.java que tiene como responsabilidades recibir el request del gRPC, abrir la conexión de la base de datos por medio del archivo Db en su función getConnection, llama a las funciones del sistema cliente o admin maneja las respectivas transacciones de ambos, maneja las transacciones y evita errores de Grpc y de java.



Luego está la carpeta util para el manejo de excepciones



En este se encuentra AppException.java enfocado en los errores de java en general.



En esta carpeta también se encuentran los errores enfocados en el GRPC traduciendo excepciones para el cliente, con la librería io.grpc.status sus principales excepciones son:

- INVALID_ARGUMENT → Status.INVALID_ARGUMENT
- NOT_FOUND → Status.NOT_FOUND
- PERMISSION_DENIED → Status.PERMISSION_DENIED
- FAILED_PRECONDITION → Status.FAILED_PRECONDITION
- INTERNAL → Status.INTERNAL

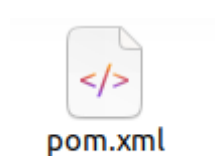
Luego esta suelto serverMain.java



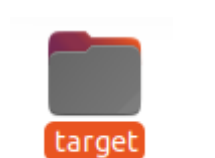
Este sirve para inicializar el servidor.



Dentro de biblioteca-service esta el pom el src y el target



El pom posee las dependencias que se usaran en el proyecto, mas a fondo estan: la version del formato del pom, dependencia para ocompilar el pom y actualizarlo automaticamente con el comando mvn compile, esta netty shaded que es el motor del servidor gRPC que posee el ´protocolo HTTP/2 en netty el protobuf que serializa los mensajes transaccionales, el stub para las clases que se van a implementar, la libreria de postgre, el build para definri como maven construye el proyecto, y extensions para detectar el sistema operativo y plugins como protoArtifact que permite hacer pipelines sirve para serializar a binario deserializar usar las clases generadas del proto, construir mensajes con builder manejar los enums y datos.



En el target se genera todo lo del maven generando estas 4 carpetas



El target crea el maven automaticamente cuando se compila que contiene todas las instancias clases compiladas para convertirse en bytecode, luego esta el plugin que es lo que se ejecuta levantando las conexiones los .exe}, el protocolo dependencies posee archivos de dependencias externas(del protobuf para que funcione) , en generated sources se encuentran los archivo.java generados por el contrato.proto,

3.4.3 COMPONENTE CLIENTE

Creamos antes la carpeta del proyecto

```

nte@MIG441:~$ cd Documents
nte@MIG441:~/Documents$ mkdir -p ~/cliente-python
nte@MIG441:~/Documents$ ls
nte@MIG441:~/Documents$ mkdir -p cliente-python
nte@MIG441:~/Documents$ ls
cliente-python
nte@MIG441:~/Documents$ cd cliente-python

```

Para el cliente instalamos las dependencias de python para correr el codigo en python. Luego desde el cliente hacemos el comando para pegar el archivo .proto en el cliente

```

estudiante@10.43.100.90: Permission denied (publickey,password).
estudiante@MIG387:~$ scp estudiante@10.43.100.90:/home/estudiante/Documents/a/biblioteca-server/src/main/proto/biblioteca.proto .
estudiante@10.43.100.90's password:
biblioteca.proto                                100% 1715    338.2KB/s   00:00
estudiante@MIG387:~$

```

Copia archivos con protocolo tls de un pc a el otro

```

estudiante@MIG441:~/Documents/cliente-python$ scp estudiante@10.43.100.90:/home/estudiante/Documents/a/biblioteca-server/src/main/p
teca.proto .
The authenticity of host '10.43.100.90 (10.43.100.90)' can't be established.

```

Luego instalamos python

```

on$ python3 -m pip install --user grpcio grpcio-tools
home/estudiante/.local/lib/python3.10/site-packages (1.

```

Luego generamos los stubs del proyecto (los documentos para exec)

```
estudiante@MIG441:~/Documents/cliente-python$ python3 -m grpc_tools.protoc  
--python_out=. \  
--grpc_python_out=. \  
biblioteca.proto  
estudiante@MIG441:~/Documents/cliente-python$ ls  
biblioteca_pb2_grpc.py biblioteca_pb2.py biblioteca.proto
```

Luego creamos el código de cliente.py que posee la lógica del cliente traducida a python.

El código presentado corresponde al cliente del sistema distribuido de biblioteca desarrollado utilizando gRPC. Su función principal es permitir la comunicación entre el usuario y el servidor, donde realmente se ejecuta la lógica del negocio. Este cliente se conecta a una dirección IP y puerto específicos, estableciendo un canal de comunicación que permite realizar llamadas a procedimientos remotos. A través de este mecanismo, el sistema puede ejecutar funciones en otra máquina como si fueran funciones locales, lo que constituye la base del modelo cliente-servidor en sistemas distribuidos.

La conexión se establece mediante la creación de un canal gRPC y un objeto denominado stub. El stub actúa como un intermediario que permite invocar los métodos definidos en el archivo .proto. Cada vez que el usuario selecciona una opción del menú, el cliente construye un mensaje de solicitud y lo envía al servidor. El servidor procesa la petición, ejecuta la lógica correspondiente (como consultar libros o registrar un préstamo) y devuelve una respuesta que el cliente recibe y muestra en pantalla.

El programa está organizado en un menú principal que permite elegir entre modo cliente o modo administrador. El modo cliente ofrece funcionalidades como consultar libros disponibles, buscar un libro por ISBN, solicitar un préstamo, devolver un libro y visualizar préstamos activos. Por otro lado, el modo administrador permite realizar operaciones más avanzadas como cargar la base de datos, agregar o eliminar libros y consultar el estado completo de la biblioteca. Estas acciones también se ejecutan mediante llamadas remotas, garantizando que toda la lógica central permanezca en el servidor.

Adicionalmente, el código incluye funciones auxiliares encargadas únicamente de mostrar en consola la información recibida desde el servidor. Estas funciones no

realizan procesamiento complejo, sino que organizan y presentan los datos de manera clara para el usuario. También se implementa manejo de errores mediante la captura de excepciones `RpcError`, lo cual es fundamental en sistemas distribuidos debido a que la comunicación en red puede presentar fallos.

En conclusión, este cliente demuestra la implementación práctica de conceptos básicos vistos en la materia de Introducción a Sistemas Distribuidos, tales como la comunicación remota mediante RPC, la separación entre cliente y servidor, el uso de middleware (gRPC) y la interacción entre diferentes nodos a través de la red. Aunque el sistema no implementa características avanzadas como balanceo de carga o tolerancia a fallos, sí evidencia el funcionamiento fundamental de un sistema distribuido en un entorno controlado y académico.

4 diseño del servicio gRPC

4.1 Definición del servicio gRPC

El diseño del servicio remoto se fundamenta en la definición de un archivo `.proto`, el cual actúa como contrato formal entre clientes y servidor. Este archivo define de manera independiente del lenguaje los métodos disponibles, las estructuras de datos intercambiadas y el tipo de comunicación establecida.

El uso de Protocol Buffers permite establecer tipificación fuerte, validación estructural y compatibilidad entre lenguajes, garantizando interoperabilidad entre el cliente desarrollado en Python y el servidor implementado en Java 17.

El archivo `.proto` constituye la base del sistema, ya que a partir de él se genera automáticamente:

- El código stub cliente en Python.
- Las clases base del servicio en Java.
- Las estructuras de mensajes serializables.

Esto elimina ambigüedades en la comunicación y asegura consistencia estructural entre ambas plataformas.

4.2 Definición de Servicio Remoto

Dentro del archivo `.proto` se define el servicio principal del sistema, el cual agrupa las operaciones disponibles para la gestión de la biblioteca.

El servicio incluye métodos remotos para:

- Registrar usuarios
- Registrar libros
- Realizar préstamos
- Consultar libros
- Devolver libros

Cada método está definido bajo el modelo request–response, lo que implica que el cliente envía una solicitud estructurada y recibe una respuesta igualmente estructurada.

Este diseño síncrono facilita el control del flujo de ejecución y permite verificar inmediatamente el resultado de cada operación.

4.3 Diseño de los Mensajes

Los mensajes definidos en el archivo `.proto` representan las estructuras de datos intercambiadas entre cliente y servidor. Cada mensaje contiene campos fuertemente tipados, tales como:

- Identificadores de usuario
- ISBN de libros
- Fechas
- Estados
- Mensajes de respuesta
- Indicadores de éxito o error

La tipificación estricta impuesta por Protocol Buffers reduce errores en tiempo de ejecución y garantiza que los datos enviados cumplan con el formato esperado.

Además, al tratarse de un formato binario compacto, Protocol Buffers optimiza el tamaño de los mensajes transmitidos, mejorando la eficiencia en red.

4.4 Generación de Código Automático

Una vez definido el archivo `.proto`, se utilizó el compilador `protoc` junto con los plugins correspondientes para generar automáticamente el código base tanto para Java como para Python.

En el servidor Java se generaron:

- Clases base del servicio.
- Clases de mensajes.
- Interfaces para implementación de métodos RPC.

En el cliente Python se generaron:

- Stubs para invocar métodos remotos.
- Clases de mensajes serializables.

Este proceso automatizado garantiza que ambas partes compartan exactamente la misma definición estructural del servicio, eliminando inconsistencias y errores de interpretación.

4.5 Tipo de Comunicacion Implementada

El sistema implementa comunicación RPC síncrona sobre HTTP/2. Esto significa que cada cliente realiza una invocación remota y espera la respuesta del servidor antes de continuar con la ejecución.

No se implementaron mecanismos de streaming bidireccional ni asincronía avanzada, ya que el alcance del taller está enfocado en operaciones transaccionales simples y controladas.

Sin embargo, el uso de gRPC sobre HTTP/2 permite que el servidor maneje múltiples solicitudes concurrentes internamente gracias al modelo de ejecución multihilo del framework, aunque no se implementó control explícito de concurrencia a nivel de lógica de negocio.

4.6 Justificacion del uso de gRPC

La elección de gRPC frente a otras tecnologías como REST o Java RMI se fundamenta en:

- Mayor eficiencia en serialización (Protocol Buffers vs JSON).
- Mejor rendimiento en comunicación gracias a HTTP/2.
- Soporte nativo para múltiples lenguajes.
- Generación automática de código cliente y servidor.
- Tipificación fuerte del contrato de comunicación.

Esto permite demostrar interoperabilidad real entre entornos heterogéneos (Java y Python), uno de los objetivos principales del taller.

5 Diseño de la Base de Datos

5.1 Justificación del modelo Relacional

El sistema implementado requiere persistencia estructurada de información relacionada con usuarios, libros y préstamos, lo cual implica la necesidad de un modelo de datos que garantice consistencia, integridad referencial y control transaccional. Para este propósito se seleccionó PostgreSQL 14.20 como motor de base de datos relacional, debido a su robustez, soporte para transacciones ACID, manejo avanzado de restricciones y capacidad de configuración para entornos distribuidos.

El modelo relacional fue diseñado bajo principios de normalización, evitando redundancia innecesaria y asegurando que cada entidad represente un concepto claramente delimitado dentro del dominio del sistema. La estructura permite mantener coherencia entre los estados de los libros y los registros de préstamo, garantizando que no existan inconsistencias entre la información almacenada y la lógica del negocio implementada en el servidor.

5.2 Modelo General de Entidades

La base de datos denominada biblioteca está compuesta por tres entidades principales:

- usuario
- libro
- prestamo

Estas entidades representan los actores y eventos fundamentales del sistema. La relación entre ellas se establece mediante claves foráneas que permiten modelar correctamente el vínculo entre un usuario y los libros que toma en préstamo.

El modelo puede interpretarse de la siguiente manera:

- Un usuario puede tener múltiples préstamos.
- Un libro puede estar asociado a múltiples préstamos históricos.
- Un préstamo relaciona exactamente un usuario con un libro.

Este diseño respeta el principio de dependencia funcional y mantiene claridad semántica en la representación de datos.

5.3 Diseño de la Entidad Usuario

La tabla usuario almacena la información básica de los usuarios registrados en el sistema.

Campos principales:

- id_usuario: Identificador único autoincremental (clave primaria).
- nombre: Nombre del usuario.
- apellido: Apellido del usuario.
- email: Correo electrónico único.
- telefono: Número de contacto.

El campo id_usuario se define como clave primaria, garantizando unicidad e identificación inequívoca de cada registro. El campo email posee una restricción UNIQUE para evitar duplicidades y reforzar la integridad de la información.

Las restricciones NOT NULL aseguran que los datos esenciales estén siempre presentes, evitando registros incompletos que puedan afectar la lógica de negocio.

5.4 Diseño de la entidad Libro

La tabla libro representa el catálogo bibliográfico gestionado por el sistema.

Campos principales:

- isbn: Identificador único del libro (clave primaria).
- titulo: Título del libro.
- autor: Autor del libro.
- editorial: Editorial asociada.
- anio_publicacion: Año de publicación.
- estado: Tipo ENUM que representa el estado actual del libro.

El ISBN fue seleccionado como clave primaria debido a que representa un identificador estándar único en sistemas bibliográficos. El uso de un tipo ENUM para el campo estado permite controlar estrictamente los valores posibles (por ejemplo: disponible, prestado, etc.), evitando inconsistencias semánticas en los datos.

Además, se incorporan restricciones CHECK para validar coherencia en el año de publicación, asegurando que los valores almacenados sean lógicamente válidos.

5.5 Diseño de la Entidad Prestamo

La tabla prestamo modela la relación transaccional entre usuarios y libros.

Campos principales:

- `id_prestamo`: Identificador único autoincremental (clave primaria).
- `id_usuario`: Clave foránea que referencia a usuario.
- `isbn`: Clave foránea que referencia a libro.
- `fecha_prestamo`: Fecha en que se realiza el préstamo.
- `fecha_devolucion`: Fecha estimada o real de devolución.
- `estado`: Tipo ENUM que indica el estado del préstamo (activo, devuelto, etc.).

La entidad préstamo cumple un papel fundamental en la consistencia del sistema, ya que registra cada operación realizada por los usuarios y permite llevar trazabilidad histórica.

Se definieron claves foráneas con restricciones ON UPDATE CASCADE y ON DELETE RESTRICT, lo cual garantiza que:

- No se eliminen usuarios o libros que tengan préstamos asociados.
- Se mantenga coherencia ante actualizaciones.

5.6 Integridad Referencial y Restricciones

El diseño incorpora múltiples mecanismos de control para garantizar integridad:

- Claves primarias en cada entidad.
- Claves foráneas para mantener relaciones válidas.
- Restricciones UNIQUE.
- Restricciones CHECK.
- Tipos ENUM personalizados.
- Índices para optimización de consultas.

Particularmente relevante es la restricción única parcial definida sobre la tabla préstamo, que impide que un mismo libro tenga más de un préstamo activo simultáneamente. Esta decisión de diseño refuerza la consistencia incluso si el servidor no implementa control explícito de concurrencia.

De esta manera, la base de datos actúa como una capa adicional de protección ante posibles inconsistencias lógicas.

5.7 Optimización e índices

Se crearon índices sobre campos frecuentemente consultados, tales como:

- Estado del préstamo.
- Identificador de usuario en la tabla préstamo.
- ISBN en condiciones específicas.

Estos índices permiten mejorar el rendimiento de consultas relacionadas con disponibilidad de libros y validación de préstamos activos, reduciendo tiempos de respuesta en operaciones remotas.

5.8 Configuración para Acceso Remoto

Dado que la base de datos se encuentra en una máquina virtual independiente, fue necesario configurar PostgreSQL para aceptar conexiones remotas. Esto implicó:

- Modificar el archivo `postgresql.conf` estableciendo `listen_addresses = '*'`.
- Configurar el archivo `pg_hba.conf` para permitir autenticación desde la IP del servidor Java.
- Reiniciar el servicio PostgreSQL.
- Verificar apertura del puerto 5432.

Esta configuración permitió que el servidor Java, ubicado en una máquina distinta, pudiera establecer conexión mediante JDBC de forma segura y controlada.

6 Implementación del Servicio

6.1 Estructura y Configuración del taller

El servidor fue desarrollado utilizando **Java 17** como lenguaje principal y **Maven 3.6.3** como herramienta de gestión de dependencias y automatización de compilación. Esta elección permite garantizar compatibilidad con las bibliotecas modernas de gRPC y mantener una estructura de taller organizada bajo estándares industriales.

El archivo `pom.xml` cumple un papel central dentro del sistema, ya que define todas las dependencias necesarias para la ejecución del servidor. Entre las más relevantes se encuentran:

- `grpc-netty-shaded` para el transporte HTTP/2.
- `grpc-protobuf` y `grpc-stub` para la serialización y generación de código.
- Driver JDBC oficial de PostgreSQL.
- Plugins de compilación para Protocol Buffers.
- Configuración de versión de Java (17).

La definición del contrato del servicio se encuentra en el archivo `.proto`, ubicado en el directorio `src/main/proto`. A partir de este archivo, Maven genera automáticamente las clases Java necesarias para la comunicación remota, incluyendo:

- Clases de mensajes.
- Clases stub.
- Clase base abstracta del servicio.

6.2 Arquitectura Interna por Capas

El servidor fue diseñado siguiendo una arquitectura multicapa que separa claramente las responsabilidades del sistema. Esta organización mejora la mantenibilidad, la escalabilidad y la claridad estructural.

La estructura interna se compone de las siguientes capas:

- ServerMain
- Service
- Business
- Repository
- DB
- Util

ServerMain

Es el punto de entrada del sistema. En esta clase se configura el puerto 50051 y se registra la implementación del servicio gRPC. El servidor se construye utilizando `ServerBuilder.forPort(50051)` y se mantiene en ejecución constante esperando solicitudes entrantes.

Capa Service

Implementa la clase base generada por gRPC a partir del archivo `.proto`. Su función es actuar como intermediario entre la red y la lógica interna del sistema.

Sus responsabilidades incluyen:

- Recibir solicitudes en formato Protocol Buffers.
- Extraer los datos del request.
- Invocar la capa de negocio.
- Construir y enviar la respuesta.
- Manejar errores técnicos controlados.

Esta capa no contiene reglas de negocio complejas.

Capa Business

Representa el núcleo lógico del sistema. Aquí se implementan las reglas que determinan cómo debe comportarse el sistema ante cada operación.

Por ejemplo:

- Verificar que el usuario exista.
- Validar disponibilidad del libro.
- Controlar préstamos activos.
- Coordinar operaciones transaccionales.

La capa Business orquesta el flujo completo de cada operación y centraliza las decisiones críticas del sistema.

Capa Repository

Contiene las clases encargadas de interactuar directamente con PostgreSQL mediante consultas SQL. Cada entidad posee su repositorio correspondiente.

Sus responsabilidades:

- Ejecutar consultas SELECT.
- Ejecutar INSERT y UPDATE.
- Retornar resultados estructurados.

No contiene lógica de negocio, únicamente acceso a datos.

Capa DB

Encapsula la configuración de la conexión JDBC hacia la base de datos remota. Aquí se definen:

- URL de conexión.
- Usuario.
- Contraseña.
- Creación de objetos Connection.

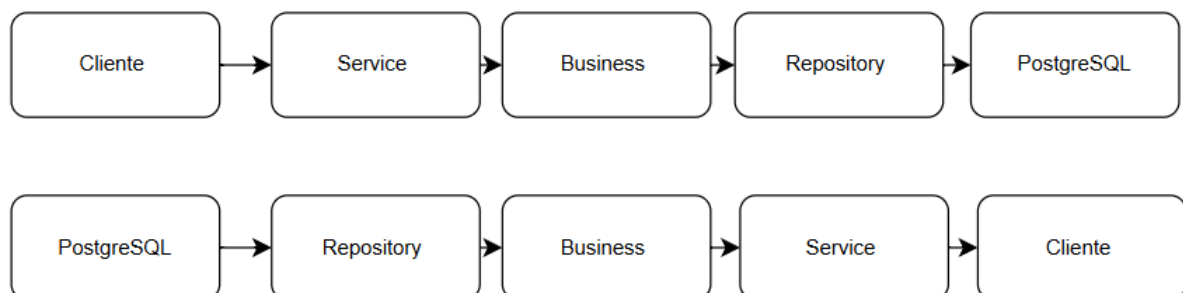
Esto permite desacoplar la lógica del sistema de los detalles técnicos de conexión.

Capa Util

Gestiona el manejo estructurado de excepciones. Permite capturar errores y convertirlos en respuestas adecuadas para el cliente sin interrumpir la ejecución del servidor.

6.3 Flujo de Procesamiento de una Solicitud

El flujo interno del servidor puede describirse como una cadena estructurada de responsabilidades:



Para ejemplificar, en el caso de un registro de préstamo el flujo sería:

1. El cliente envía la solicitud mediante gRPC.
2. La capa Service recibe el request.
3. Se extraen los datos necesarios (ID de usuario, ISBN).
4. La capa Business valida:
 - a. Existencia del usuario.
 - b. Disponibilidad del libro.
5. Si las validaciones son correctas, se delega a los repositorios:
 - a. Insertar nuevo préstamo.
 - b. Actualizar estado del libro.
6. Se construye la respuesta.
7. Se envía al cliente.

Este flujo demuestra separación clara de responsabilidades y coherencia estructural.

6.4 Concurrencia y Robustez

El servidor utiliza el modelo multihilo interno de gRPC, lo cual permite atender múltiples solicitudes simultáneamente sin bloquear el procesamiento.

Aunque no se implementaron mecanismos explícitos de control de concurrencia a nivel de código, la integridad de los datos se garantiza mediante:

- Restricciones de base de datos.
- Claves foráneas.
- Restricciones únicas parciales.
- Validaciones previas en la capa Business.

El manejo estructurado de excepciones evita fallos inesperados y permite retornar mensajes de error controlados al cliente.

El servidor permanece en ejecución continua y únicamente se detiene mediante interrupción manual o error crítico no controlado.

7 Implementacion del Cliente

7.1 Entorno y Configuracion del Cliente

El cliente fue desarrollado en **Python 3**, utilizando la biblioteca oficial de gRPC para este lenguaje. La elección de Python permite demostrar interoperabilidad entre lenguajes distintos, ya que el servidor está implementado en Java y el cliente en Python.

Las dependencias principales instaladas fueron:

- grpcio
- grpcio-tools
- protobuf

La instalación se realizó mediante:

```
pip install grpcio grpcio-tools
```

A partir del archivo .proto, se generaron automáticamente los archivos necesarios para el cliente utilizando el compilador de Protocol Buffers. Esto generó:

- El archivo de mensajes (*_pb2.py)
- El archivo stub (*_pb2_grpc.py)

Estos archivos permiten que el cliente construya solicitudes y se comuniquen con el servidor de manera tipada y estructurada.

7.2 Conexion al Servidor

El cliente establece conexión mediante un canal gRPC hacia la dirección IP del servidor y el puerto 50051.

El uso de `insecure_channel` indica que la comunicación se realiza sin cifrado TLS, adecuado para entornos controlados de laboratorio o pruebas académicas.

Una vez creado el canal, se instancia el stub generado automáticamente, el cual permite invocar los métodos remotos definidos en el contrato .proto.

Este mecanismo elimina la necesidad de manejar manualmente sockets o protocolos HTTP, ya que gRPC abstrae completamente esa complejidad.

7.3 Invocacion de Metodos Remotos

El cliente implementa funciones específicas para cada operación disponible en el sistema, tales como:

- Registrar usuario
- Registrar libro
- Consultar libro
- Registrar préstamo
- Registrar devolución

Cada operación sigue el siguiente patrón:

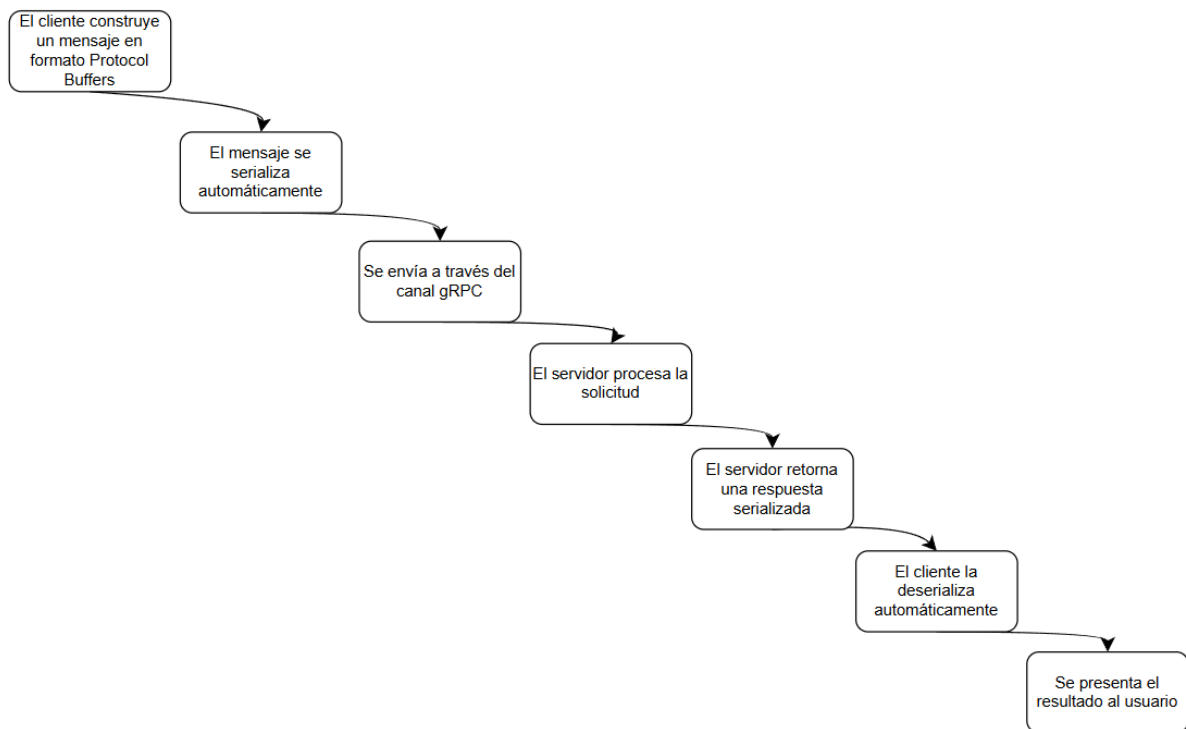
1. Construcción del objeto request.
2. Invocación del método remoto mediante el stub.
3. Recepción de la respuesta.
4. Impresión o procesamiento del resultado.

Por ejemplo, en el registro de un préstamo:

- Se crea el objeto PrestamoRequest con los datos necesarios.
- Se invoca `stub.registrarPrestamo(request)`.
- Se recibe una respuesta estructurada.
- Se muestra el mensaje al usuario.

Este esquema mantiene coherencia con el contrato definido en el archivo `.proto`.

7.4 Flujo de Comunicacion Cliente-Servidor



- El cliente construye un mensaje en formato Protocol Buffers.
- El mensaje se serializa automáticamente.
- Se envía a través del canal gRPC.
- El servidor procesa la solicitud.
- El servidor retorna una respuesta serializada.
- El cliente la deserializa automáticamente.
- Se presenta el resultado al usuario.

Todo este proceso ocurre de forma transparente gracias al framework gRPC, que maneja la serialización, el transporte HTTP/2 y la deserialización sin intervención manual del desarrollador.

7.5 Manejo de Errores en el Cliente

El cliente implementa bloques de manejo de excepciones para capturar posibles errores durante la comunicación, tales como:

- Fallo de conexión al servidor.
- Timeout.
- Respuestas inválidas.
- Errores generados por el servidor.

Estos errores se capturan utilizando excepciones del tipo `grpc.RpcError`, permitiendo mostrar mensajes claros al usuario y evitar que la aplicación termine abruptamente.

Este manejo fortalece la robustez del sistema distribuido.

7.6 Interoperabilidad entre Lenguajes

Uno de los aspectos más relevantes del taller es la interoperabilidad demostrada entre Java y Python. Gracias a Protocol Buffers, ambos lenguajes comparten un contrato común definido en el archivo `.proto`.

Esto permite que:

- El servidor Java procesa solicitudes enviadas por un cliente Python.
- No exista dependencia del lenguaje en la comunicación.
- El sistema sea escalable hacia otros lenguajes compatibles con gRPC.

Este punto es clave en la sustentación, ya que demuestra comprensión de arquitecturas distribuidas modernas.

8 Pruebas y validacion del sistema

8.1 Verificacion de Infraestructura y Conectividad de Red

Antes de ejecutar cualquier prueba funcional, se realizó la validación de la infraestructura distribuida para garantizar que cada una de las máquinas virtuales pudiera comunicarse correctamente dentro de la red. Esta etapa es fundamental en un sistema distribuido, ya que la disponibilidad y conectividad entre nodos condiciona el correcto funcionamiento de los servicios remotos.

Se verificó la conectividad mediante pruebas de ping entre las máquinas cliente, servidor y base de datos, confirmando una pérdida de paquetes del 0%, lo que indica estabilidad en la red virtual configurada. Posteriormente, se comprobó que el servidor gRPC estuviera escuchando correctamente en el puerto 50051 y que PostgreSQL estuviera habilitado para conexiones remotas en el puerto 5432. Esta validación permitió asegurar que la arquitectura física definida en el diseño se encontraba operativa antes de realizar pruebas lógicas o funcionales.

```
estudiante@MIG441:~/Documents/cliente-python$ ping 10.43.100.90
PING 10.43.100.90 (10.43.100.90) 56(84) bytes of data.
64 bytes from 10.43.100.90: icmp_seq=1 ttl=64 time=2.61 ms
64 bytes from 10.43.100.90: icmp_seq=2 ttl=64 time=0.392 ms
64 bytes from 10.43.100.90: icmp_seq=3 ttl=64 time=0.487 ms
64 bytes from 10.43.100.90: icmp_seq=4 ttl=64 time=0.359 ms
64 bytes from 10.43.100.90: icmp_seq=5 ttl=64 time=0.362 ms
```

```
estudiante@MIG387:~/Documents/a/biblioteca-server$ ping 10.43.99.128
PING 10.43.99.128 (10.43.99.128) 56(84) bytes of data.
64 bytes from 10.43.99.128: icmp_seq=1 ttl=64 time=0.533 ms
64 bytes from 10.43.99.128: icmp_seq=2 ttl=64 time=0.448 ms
64 bytes from 10.43.99.128: icmp_seq=3 ttl=64 time=0.385 ms
64 bytes from 10.43.99.128: icmp_seq=4 ttl=64 time=0.485 ms
64 bytes from 10.43.99.128: icmp_seq=5 ttl=64 time=0.359 ms
```

Estas pruebas no solo confirman la existencia de comunicación entre nodos, sino que evidencian que el sistema cumple con el principio fundamental de separación física propio de las arquitecturas distribuidas.

```
estudiante@MIG387:~$ ss -tulnp | grep 50051
tcp    LISTEN 0      4096      *:50051      *:~          users:((("java",pid=460522,fd=95))
```

8.2 Inicializacion y Disponibilidad del Servidor gRPC

Una vez validada la conectividad de red, se procedió a iniciar el servidor desarrollado en Java 17 utilizando Maven como herramienta de gestión y ejecución. El servidor fue desplegado correctamente y quedó a la espera de solicitudes remotas provenientes de los clientes implementados en Python.

```
estudiante@MIG187:~$ sudo ss -tulnp | grep 5432
tcp    LISTEN 0      244      0.0.0.0:5432  0.0.0.0:*    users:((("postgres",pid=328932,fd=5))
tcp    LISTEN 0      244      [::]:5432  [::]:*       users:((("postgres",pid=328932,fd=6))
```

El inicio exitoso del servidor confirma que el archivo .proto fue correctamente compilado, que las clases generadas por Protocol Buffers están integradas adecuadamente y que la configuración del servicio gRPC es funcional. Además, esta etapa valida que la estructura por capas implementada (comunicación, lógica de negocio, acceso a datos y conexión a base de datos) se encuentra correctamente integrada.

La correcta inicialización del servidor representa la activación del núcleo del sistema distribuido, ya que centraliza la lógica que coordina las operaciones entre cliente y base de datos.

8.3 Validacion del Modelo Relacional en PostgreSQL

Posteriormente, se verificó el correcto despliegue del modelo relacional en la máquina virtual destinada a la base de datos. Se comprobó la existencia de las tablas usuario, libro y prestamo, así como de los tipos ENUM definidos para el manejo de estados.

```
biblioteca=# \dt
               List of relations
 Schema | Name      | Type  | Owner
-----+-----+-----+-----
 public | libro     | table | postgres
 public | prestamo  | table | postgres
 public | usuario   | table | postgres
(3 rows)
```

Mediante comandos de inspección de estructura (\d y \dT+), se validó la presencia de claves primarias, claves foráneas, restricciones CHECK, índices y restricciones UNIQUE. Esta revisión permitió confirmar que el modelo de datos no solo fue creado correctamente, sino que incorpora mecanismos de integridad referencial que garantizan consistencia en la información almacenada.

```
biblioteca=# \dT prestamo
Table "public.prestamo"
  Column      | Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
 id_prestamo  | integer       |           | not null | nextval('prestamo_id_prestamo_seq'::regclass)
 id_usuario   | integer       |           | not null |
 isbn         | character varying(20) |         | not null |
 fecha_prestamo | date          |           | not null | CURRENT_DATE
 fecha_devolucion | date          |           | not null |
 estado       | estado_prestamo |         | not null | 'activo'::estado_prestamo
Indexes:
    "prestamo_pkey" PRIMARY KEY, btree (id_prestamo)
    "idx_prestamo_estado" btree (estado)
    "idx_prestamo_usuario" btree (id_usuario)
    "uq_prestamo_isbn_activo" UNIQUE, btree (isbn) WHERE estado = 'activo'::estado_prestamo
Check constraints:
    "chk_prestamo_fechas" CHECK (estado = 'activo'::estado_prestamo AND fecha_devolucion IS NULL OR estado = 'devuelto'::estado_prestamo AND fecha_devolucion IS NOT NULL)
Foreign-key constraints:
    "fk_prestamo_libro" FOREIGN KEY (isbn) REFERENCES libro(isbn) ON UPDATE CASCADE ON DELETE RESTRICT
    "fk_prestamo_usuario" FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario) ON UPDATE CASCADE ON DELETE RESTRICT
```

La validación estructural de la base de datos es clave en sistemas distribuidos, ya que la persistencia de datos constituye el punto final de cada operación remota ejecutada desde los clientes.

8.4 Prueba Funcionales del Servicio Remoto

Con la infraestructura y el servidor operativos, se realizaron pruebas funcionales ejecutando operaciones desde los clientes Python hacia el servidor Java. Estas pruebas incluyeron:

- Consulta de libros disponibles.
- Búsqueda de libros por criterios específicos.
- Registro de préstamos.
- Consulta de préstamos activos.
- Devolución de libros.

- Registro de nuevos libros.
- Eliminación de libros existentes.


```
estudiante@MIG441:~/Documents/cliente-python$ python3 cliente.py
```

```
===== SISTEMA BIBLIOTECA =====
```

- 1) Cliente
- 2) Admin
- 0) Salir

Opción: 1

```
--- MENU CLIENTE ---
```

- 1) Ver libros
- 2) Buscar libro
- 3) Pedir préstamo
- 4) Devolver libro
- 5) Ver mis préstamos
- 0) Volver

Opción: 4

ID préstamo: 12

Error: Préstamo activo no existe

```
--- MENU CLIENTE ---
```

- 1) Ver libros
- 2) Buscar libro
- 3) Pedir préstamo
- 4) Devolver libro
- 5) Ver mis préstamos
- 0) Volver

Opción: 4

ID préstamo: 1

Devolución realizada

```
--- MENU CLIENTE ---
1) Ver libros
2) Buscar libro
3) Pedir préstamo
4) Devolver libro
5) Ver mis préstamos
0) Volver
Opción: 3
ID usuario: 1
ISBN:
Error: ISBN vacío

--- MENU CLIENTE ---
1) Ver libros
2) Buscar libro
3) Pedir préstamo
4) Devolver libro
5) Ver mis préstamos
0) Volver
Opción: 0

===== SISTEMA BIBLIOTECA =====
1) Cliente
2) Admin
0) Salir
Opción: 2
Clave admin: admin

--- MENU ADMIN ---
1) Cargar base de datos
2) Agregar libro
3) Eliminar libro
4) Ver biblioteca
5) Ver libros repetidos
0) Volver
Opción: 1
Carga/seed OK. Libros insertados: 0
```

Cada una de estas operaciones implicó un flujo completo dentro del sistema: el cliente envió una solicitud gRPC, el servidor procesó la lógica correspondiente, se ejecutaron consultas SQL sobre PostgreSQL y finalmente se retornó una respuesta serializada al cliente.

El correcto funcionamiento de estas operaciones demuestra que el contrato definido en el archivo .proto es consistente, que existe interoperabilidad real entre Java y Python y que la serialización mediante Protocol Buffers garantiza integridad en el intercambio de datos.

8.5 Validación de Reglas de Negocio

Durante las pruebas también se verificó que el sistema respete las reglas de negocio definidas. Entre ellas se destacan:

- Un libro no puede tener más de un préstamo activo simultáneamente.
- El ISBN debe ser único.
- El año de publicación debe cumplir con las restricciones definidas.
- Un préstamo no puede registrarse para un usuario inexistente.
- No es posible devolver un préstamo que no esté activo.

```

--- MENU ADMIN ---
1) Cargar base de datos
2) Agregar libro
3) Eliminar libro
4) Ver biblioteca
5) Ver libros repetidos
0) Volver
Opción: 2
ISBN: 13
Título: trece
Autor: erce
Editorial: ra
Año: 2029
Error: Error interno: ERROR: new row for relation "libro" violates check constraint "chk_anio_publicacion"
Detail: Failing row contains (13, trece, erce, ra, 2029, disponible).

--- MENU ADMIN ---
1) Cargar base de datos
2) Agregar libro
3) Eliminar libro
4) Ver biblioteca
5) Ver libros repetidos
0) Volver
Opción: 2
ISBN: 12
Título: ter
Autor: eed
Editorial: er
Año: 2020
Libro agregado

```

Estas validaciones se ejecutan tanto a nivel de lógica de negocio en el servidor como a nivel de restricciones en la base de datos, lo que proporciona una doble capa de protección ante inconsistencias.

La comprobación de estas reglas evidencia que el sistema no solo ejecuta operaciones básicas, sino que mantiene coherencia semántica en el dominio del problema.

8.6 Pruebas de Manejo de errores y robustez

Se realizaron pruebas introduciendo datos inválidos o escenarios no permitidos con el objetivo de evaluar la robustez del sistema. Entre los errores detectados y correctamente gestionados se encuentran:

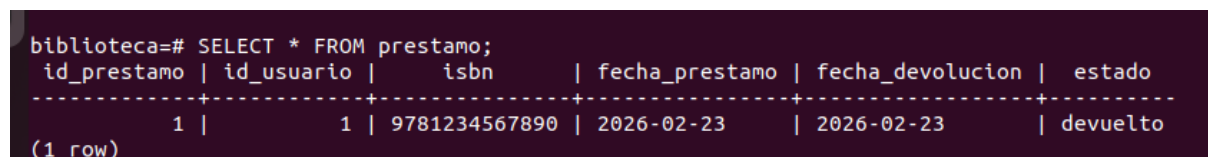
- Intentos de insertar registros duplicados.
- Violaciones de restricciones CHECK.
- Valores fuera de rango.
- ISBN vacíos.
- Préstamos inexistentes.

En todos los casos, el sistema respondió de manera controlada, generando mensajes de error coherentes sin comprometer la estabilidad del servidor ni la integridad de la base de datos.

Esto demuestra que el sistema cuenta con un manejo estructurado de excepciones, lo cual es especialmente importante en arquitecturas distribuidas donde los fallos pueden originarse en diferentes capas.

8.7 Verificación de Persistencia de Datos

Finalmente, se ejecutaron consultas directas en PostgreSQL para confirmar que las operaciones realizadas desde los clientes realmente modificaban el estado de la base de datos. Se validó la inserción de usuarios, libros y préstamos, así como la actualización del estado de los registros tras las devoluciones.



```
biblioteca=# SELECT * FROM prestamo;
```

id_prestamo	id_usuario	isbn	fecha_prestamo	fecha_devolucion	estado
1	1	9781234567890	2026-02-23	2026-02-23	devuelto

(1 row)

Esta verificación confirma que el sistema no opera únicamente a nivel de simulación, sino que existe persistencia real de datos en un nodo independiente, cumpliendo con el diseño distribuido planteado desde el inicio del taller.

8.8 Resultados Generales

Las pruebas realizadas permiten concluir que el sistema distribuido funciona de manera consistente bajo el esquema cliente-servidor implementado con gRPC. La interoperabilidad entre lenguajes, la correcta configuración de la base de datos remota y la separación por capas dentro del servidor evidencian una arquitectura sólida y estructurada.

El sistema cumple con los objetivos planteados, demostrando que es posible integrar múltiples tecnologías en entornos heterogéneos manteniendo coherencia, integridad y estabilidad operativa.

9 Conclusiones

El desarrollo del presente taller permitió aplicar de manera práctica los conceptos fundamentales estudiados en la asignatura Introducción a Sistemas Distribuidos, llevando la teoría vista en clase a un entorno funcional compuesto por múltiples máquinas virtuales que interactúan entre sí. A lo largo del proceso, se logró comprender que un sistema distribuido no se limita a ejecutar código en diferentes equipos, sino que implica coordinación, comunicación remota, separación de responsabilidades y control de la consistencia de los datos.

Uno de los aprendizajes más importantes fue entender el papel que cumple la comunicación remota en una arquitectura distribuida. Mediante la implementación de gRPC como mecanismo de interacción entre procesos, se evidenció cómo un cliente puede invocar

métodos que se ejecutan físicamente en otra máquina, como si fueran locales. Este modelo permitió comprender mejor conceptos como RPC, serialización de datos y contratos de servicio definidos mediante Protocol Buffers. La interoperabilidad lograda entre un servidor desarrollado en Java y clientes implementados en Python reforzó la idea de que los sistemas distribuidos modernos deben ser capaces de funcionar en entornos heterogéneos.

Asimismo, la configuración de cuatro máquinas virtuales independientes permitió visualizar de forma concreta la separación física de componentes, característica esencial de este tipo de arquitecturas. El hecho de que la base de datos estuviera alojada en una máquina distinta al servidor, y que los clientes operaran desde nodos diferentes, permitió comprender aspectos relacionados con la conectividad, la apertura de puertos, la configuración de servicios y la importancia de validar la comunicación antes de ejecutar operaciones funcionales.

En relación con la base de datos, el diseño del modelo relacional permitió aplicar principios de integridad referencial y consistencia. La implementación de claves primarias, foráneas y restricciones mostró que en un sistema distribuido la coherencia de la información no depende únicamente del código de la aplicación, sino también del diseño adecuado del almacenamiento de datos. Las pruebas realizadas demostraron que las reglas de negocio pueden ser reforzadas tanto en la lógica del servidor como en la estructura de la base de datos.

Durante la fase de pruebas, los resultados fueron en su mayoría satisfactorios. Las operaciones de consulta, préstamo y devolución de libros se ejecutaron correctamente a través del flujo completo cliente-servidor-base de datos. Además, la validación de errores permitió confirmar que el sistema responde de forma controlada ante entradas inválidas o violaciones de restricciones, lo cual es importante para mantener la estabilidad en entornos distribuidos.

Sin embargo, también se identificaron limitaciones propias del alcance del taller y del nivel introductorio de la asignatura. El sistema no implementa mecanismos avanzados de concurrencia ni estrategias complejas de escalabilidad, aspectos que forman parte de temas más profundos dentro del estudio de sistemas distribuidos. Esto permite reconocer que el taller representa una base sólida, pero no un sistema productivo completo. A futuro, sería interesante explorar la implementación de control de concurrencia más explícito, autenticación de usuarios o incluso despliegue en entornos de nube para ampliar la experiencia práctica.

En términos generales, el taller cumplió con el objetivo de demostrar cómo se estructura y funciona un sistema distribuido básico utilizando tecnologías actuales. Más allá de la funcionalidad obtenida, el principal aporte fue comprender el paradigma distribuido: entender que los componentes están separados físicamente, que la comunicación ocurre a través de la red y que la coordinación entre procesos requiere planificación y diseño.

En conclusión, esta experiencia permitió consolidar los fundamentos vistos en la asignatura, reforzando la comprensión de arquitecturas cliente-servidor, comunicación remota y

persistencia de datos en entornos distribuidos. El taller representa un primer acercamiento práctico al desarrollo de sistemas distribuidos y sienta las bases para profundizar en temas más avanzados en futuras etapas de formación académica.

10 Referencias

- <https://grpc.io/docs/languages/>
- https://www.youtube.com/watch?v=me5_oLWBa1M
- <https://www.youtube.com/watch?v=bmyVTXO10vs>